

Relatório Técnico — Cardiola Fase 7:

Coração Sob Controle

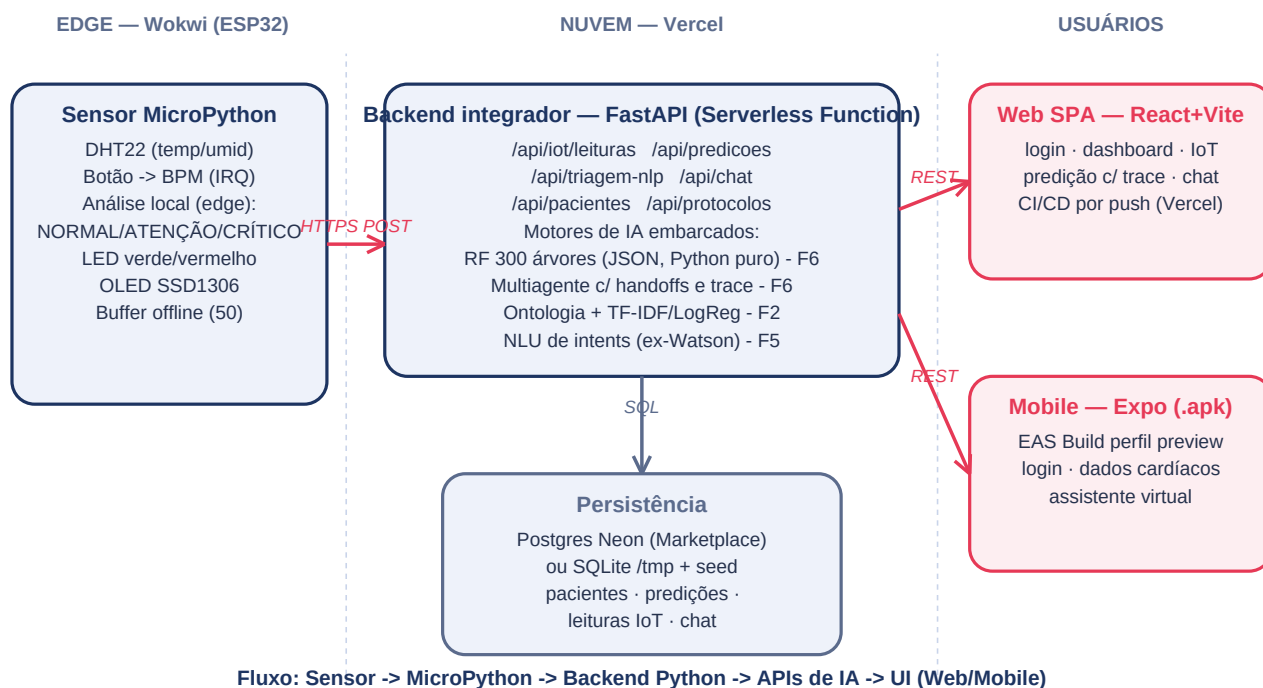
Curso de Inteligência Artificial — FIAP · Fase 7 · Junho/2026

Grupo: Tiago Martins da Silva (RM560639) · Mauricio Cortes Moreira (RM559923) · Lucas Costa dos Santos Castro (RM559625)

1. Visão geral

A Fase 7 consolida a Cardiola como **Plataforma de Inteligência Cardíaca Total**: os módulos construídos nas fases 1–6 (coleta de dados, diagnóstico NLP, monitoramento IoT, visão computacional, chatbot e sistema preditivo multiagente) foram unificados em um único produto digital com deploy profissional e CI/CD. Um **sensor ESP32 rodando MicroPython** (simulado no Wokwi) captura sinais do paciente, faz **análise local (edge)** e envia as leituras a um **backend integrador em Python**, que aciona os **motores de IA** e entrega recomendações clínicas em tempo quase-real às interfaces **Web (React+Vite na Vercel)** e **Mobile (APK via Expo EAS Build)**.

2. Diagrama de arquitetura final



3. Decisões de arquitetura

3.1 Deploy 100% Vercel. A atividade permitia combinar provedores; o grupo optou por concentrar web e backend na Vercel, com um único repositório GitHub e CI/CD automático nos dois projetos a cada push. O backend FastAPI roda como **Serverless Function Python** (*backend/api/index.py* + *rewrite* no *vercel.json*). Três adaptações tornaram isso viável:

(1) Inferência de ML em Python puro. Empacotar scikit-learn+scipy+numpy (>150 MB) numa serverless function é inviável. A Random Forest da Fase 6 (300 árvores) foi **exportada para JSON** (*ml/export_model.py*) e o serviço de ML caminha as árvores manualmente, reproduzindo *predict_proba* com **paridade verificada (delta < 1e-9)**. O classificador textual da Fase 2 (TF-IDF + Regressão Logística, 90% de acurácia em holdout) foi exportado da mesma forma (vocabulário, IDF e coeficientes). A function final tem poucos MB e cold start baixo. **(2) Persistência adaptável.** Serverless tem filesystem efêmero: a camada de dados usa **Postgres gerenciado** (Vercel Marketplace/Neon) quando *POSTGRES_URL* existe; sem ele, SQLite em */tmp* com seed idempotente a cada cold start mantém a demonstração operacional. **(3) Estado do chat em banco.** As sessões do chatbot são persistidas em tabela própria (não em memória), pois requisições podem atender em instâncias diferentes.

3.2 Conversão IoT para MicroPython (Fase 3 → 7). O firmware C/C++ da Fase 3 (Arduino: DHT22 + DS18B20 + botão, MQTT/HiveMQ) foi reescrito em MicroPython (*iot/main.py*), mantendo a essência — leitura a cada 10 s, validação de faixas de UTI e buffer local offline com sincronização — e evoluindo em três pontos: **(a)** classificação

clínica na borda (NORMAL/ATENÇÃO/CRÍTICO) antes de qualquer envio; **(b)** feedback visual — LEDs com padrões de pisca por severidade e OLED SSD1306 com BPM, temperaturas e status; **(c)** envio HTTP direto ao backend (*urequests.post*), eliminando o broker MQTT: o fluxo exigido fica mais curto e auditável. O DS18B20 foi substituído por temperatura derivada do DHT22 (+12,5 °C) por ausência de driver MicroPython nativo no Wokwi — mover o slider do DHT22 simula febre.

3.3 Unificação dos motores de IA

Motor	Fase	Implementação na Fase 7
Modelo preditivo (Random Forest, AUC 0,815)	6	Exportado p/ JSON + inferência pura; fallback heurístico calibrado
Sistema multiagente (Orquestrador → Analista de Risco → Especialista em Protocolos; tools, handoffs, trace auditável)	6	Preservado; acionado por predição manual, chat e automaticamente por leitura IoT crítica
Triagem NLP (ontologia 22 regras + risco textual)	2	nlp_service.py + endpoint /api/triagem-nlp e tela própria na web
Chatbot (intents Watson: emergência, exames, pressão, sintomas)	5	NLU local por palavras-chave — sem credenciais IBM; respostas dos dialog nodes preservadas; handoff para coleta guiada + pipeline multiagente

A integração mais relevante: uma **leitura IoT classificada como CRÍTICA pelo servidor dispara automaticamente o pipeline multiagente**, registrando predição e recomendação clínica que aparecem no dashboard — sensor e IA fechados em ciclo.

4. Experiência do usuário

Web (médico): login, dashboard com indicadores de risco e fluxo do sensor em tempo quase-real (polling 4–5 s), predição com trace completo dos agentes, triagem NLP, protocolos, métricas do modelo e chat. Hierarquia visual de risco por badges coloridos (BAIXO/MODERADO/ALTO/CRÍTICO + nível de atenção). **Mobile (paciente):** login, dashboard de dados cardíacos e assistente conversacional — APK gerado pelo perfil *preview* do EAS (*buildType: apk*), pacote *br.com.fiap.cardioia*.

5. Qualidade e reprodutibilidade

7 smoke tests (pytest) cobrem health, CRUD, predição multiagente, triagem NLP, fluxo IoT (incluindo o disparo automático da IA) e os fluxos do chat (intents da Fase 5 + triagem completa fim-a-fim). Base sintética e modelo determinísticos (seed 42); exportação com verificação automática de paridade. Repositório privado no GitHub compartilhado com o tutor; commits organizados por etapa; CI/CD ativo na Vercel.

6. Limitações e próximos passos

A autenticação é simplificada (MVP) e a base do modelo é sintética; a análise de imagens da Fase 4 está descrita como extensão natural (endpoint dedicado de visão computacional). Próximos passos: IdP real, dados clínicos anonimizados e predição assíncrona em escala. **Aviso:** sistema educacional — não substitui avaliação médica. Emergências: 192 (SAMU).